TECNOLOGO EN ANALISIS Y DESARROLLO DE SOFTWARE

SERVICIO NACIONAL DE APRENDIZAJE CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS

INFORME TÉCNICO Y RUNBOOK DE DESPLIEGUE

PROYECTO: DRIVIQUE

INTEGRANTES:

LAURA VANESSA PEREZ PERDOMO
DANNA VALENTINA BARRIOS PENAGOS
EMILY SHARITH AMEZQUITA SAAVEDRA

FICHA:3145555

2026

Tabla de contenido

1.1.4 Matriz de roles y responsables....4
1.2 Historial de Versiones del Documento....4
1.3 Resumen de Cambios (Release Notes)....5
1.3.1 Nuevas funcionalidades....5
1.3.2 Correcciones de errores (Bug fixes)....5
1.3.3 Cambios de infraestructura y dependencias....5
1.3.4 Cambios incompatibles (Breaking changes)....6
1.4 Justificación de la Migración Arquitectónica: De Servidores Gratuitos a VPS Hostinger....6
2. Alcance, Topología y Arquitectura....7
2.1 Componentes y Artefactos en Alcance....7
2.1.1 Servidor Web y Proxy Reverso Seguro (Caddy 2)....7
2.1.2 Capa Frontend Web (React 19 SPA en Contenedor Docker)....7
2.1.3 Capa Frontend Móvil (React Native Expo SDK 52 en Contenedor Docker)....7
2.1.4 Capa Backend API (Java Spring Boot 4.0.8)....7
2.1.5 Capa de Persistencia (PostgreSQL 17 Alpine en Contenedor Docker)....7
2.2 Diagrama de Arquitectura de Red e Infraestructura....7
2.3 Matriz de Ambientes, Direccionamiento IP y Puertos....8
3. Requisitos Previos y Preparación (Pre-deployment)....10
3.1 Checklist de Prerrequisitos del Sistema Operativo y VPS....10
3.1.1 Recursos de hardware disponibles....10
3.1.2 Estado del motor Docker y socket local....10
3.2 Estrategia y Ejecución de Respaldos (Backups)....10
3.2.1 Respaldo lógico de base de datos (Dump / Snapshot)....10
3.2.2 Respaldo de volúmenes persistentes de Docker....10
3.2.3 Respaldo de configuraciones de Stacks y variables....10
3.3 Gestión de Variables de Entorno y Secretos....10
3.3.1 Matriz de Variables de Entorno del Backend API (deploy/.env)....10
3.3.2 Matriz de Variables del Frontend Web (drivique.tech)....11
3.3.3 Matriz de Variables del Frontend Móvil (app.drivique.tech)....12
3.3.4 Configuración de Pasarela de Pago Wompi (Sandbox)....12

3.4 Protocolo de Comunicación y Ventana de Mantenimiento 13
4. Procedimiento de Despliegue Paso a Paso (Runbook) 14
4.1 Acceso y Administración del Servidor VPS Hostinger (SSH / Consola Web) 14
4.2 Despliegue y Migración de Base de Datos 14
4.2.1 Ejecución de scripts DDL/DML 14
4.2.2 Verificación de integridad 14
4.3 Despliegue y Orquestación del Stack de Aplicación 14
4.3.1 Corrección y Estandarización del Dockerfile (Backend) 14
4.3.2 Orquestación Integral con Docker Compose y Caddy 15
4.3.3 Automatización del Despliegue con el Script deploy.sh 15
4.4 Configuración de Enrutamiento, Dominio y Certificados SSL/TLS 15
4.4.1 Configuración de Zona DNS en Hostinger (drivique.tech) 15
4.4.2 Enrutamiento y Aprovisionamiento SSL/TLS Automático con Caddy 16
5. Plan de Verificación, Pruebas y Evidencias (Smoke Tests) 17
5.1 Matriz de Enlaces y Endpoints de Verificación de Servicios 17
5.2 Pruebas de Humo Funcionales (Smoke Tests) 17
5.2.1 Flujo de autenticación y emisión de tokens 17
5.2.2 Pruebas de lectura y consulta de datos 17
5.2.3 Pruebas de escritura y persistencia transaccional 17
5.3 Inspección de Logs y Monitoreo de Recursos 18
5.3.1 Revisión de trazas de inicio sin excepciones críticas 18
5.3.2 Monitoreo de consumo de CPU y memoria RAM 18
5.4 Registro de Evidencias Visuales (Screenshots) 18

1. Información General y Control de Versiones

1.1 Ficha Técnica del Despliegue

1.1.1 Identificación del sistema y versión (Release Tag)

- **Sistema:** Drivique Platform (Sistema Integral de Renta de Vehículos, Gestión de Flotas y Sucursales).
- **Versión / Release Tag:** v1.21.0-main (Integración completa hasta HU-INT-21).

1.1.2 Fecha, hora y ventana operativa programada

- **Fecha:** 15 de Octubre de 2026.
- **Horario / Ventana de Mantenimiento:** 08:00 – 11:30 UTC-5 (ventana de bajo impacto con 15 minutos de indisponibilidad programada).

1.1.3 Entorno de destino

- **Capa de Persistencia:** Cloud PostgreSQL 17 gestionado (PostgreSQL US-East-1) con PgBouncer Session Pooler IPv4.
- **Capa Backend API:** Cloud Web Service en Hostinger VPS / contenedor Docker en VPS Staging orquestado con Caddy 2 CE.
- **Capa Frontend Web:** Hostinger VPS Edge Network (https://drivique.tech / React 19 SPA).
- **Capa Frontend Móvil:** Hostinger VPS Web Export (https://app.drivique.tech / React Native Expo) y cliente móvil Expo Go.

1.1.4 Matriz de roles y responsables

Rol	Responsabilidad
DevOps / SysAdmin	Configuración de contenedores Docker, variables de entorno, redes Bridge y certificados SSL/TLS.
Database Administrator (DBA)	Ejecución y validación de scripts DDL/DML, esquemas, índices y pool de conexiones en PostgreSQL.
Tech Lead / Backend Lead	Auditoría de configuración del datasource, perfiles Spring Boot, CORS y control de memoria JVM.
QA Engineer	Ejecución de pruebas de humo (Smoke Tests) en Frontend Web, Frontend App y verificación de endpoints API.

1.2 Historial de Versiones del Documento

Versión	Fecha	Autor	Rol	Descripción de Modificación
1.0.0	30/10/2026	Equipo DevOps Drivique	DevOps Lead	Elaboración inicial del plan y runbook de despliegue para HU-INT-17 y HU-INT-21.

Versión	Fecha	Autor	Rol	Descripción de Modificación
1.1.0	30/10/2026	Equipo DevOps Drivique	DevOps Engineer	Inclusión de configuraciones de Frontend Web, App Móvil en Hostinger VPS, Dockerfile multi-stage y pasarela Wompi.

1.3 Resumen de Cambios (Release Notes)

1.3.1 Nuevas funcionalidades

- **Catálogo de Conductores de Entrega:** creación de la tabla location.delivery_drivers para asignación logística de entrega a domicilio/aeropuerto.
- **Capacidad y Perfiles de Sucursales:** nuevas columnas de aforo vehicular y categorización de sedes (branch-categories, branch-vehicle-capacity).
- **Réplicas de Reseñas:** capacidad de responder a calificaciones y comentarios de clientes en sucursales (add-branch-review-reply.sql).
- **Ventana de Pago Configurable:** disparador apply-reservation-rules-payment-window.sql para expiración automática de reservas no pagadas.
- **Reportes Administrativos Ampliados:** nuevos tipos de reportes y auditoría extendida (HU-INT-21).

1.3.2 Correcciones de errores (Bug fixes)

- Corrección de pantalla blanca en despliegue de Hostinger VPS mediante inyección de VITE_API_URL.
- Saneamiento y corrección de zonas horarias en cupones y promociones activas.
- Unificación de planes de cobertura a dos opciones estándar (Básica y Total).
- Validación estricta y obligatoriedad en tipos de documentos KYC para verificación de identidad.
- Solución a fallo de compilación Docker en Render mediante compilación multi-etapa con Maven.

1.3.3 Cambios de infraestructura y dependencias

- Motor de persistencia PostgreSQL 17 con extensiones pgcrypto y btree_gist.
- Integración de Liquibase 4.29 como orquestador de cambios DDL, DML y DCL.
- Dockerfile multietapa optimizado con Maven 3.9 y Eclipse Temurin 17 JRE Alpine.
- Implementación de límites de memoria JVM con Garbage Collector Serial (-Xmx384m -XX:+UseSerialGC).

1.3.4 Cambios incompatibles (Breaking changes)

- **Segregación en 10 esquemas relacionales independientes:** core, iam, location, fleet, catalog, rental, contract, billing, support, audit.
- **Validación Hibernate estricta (hibernate.ddl-auto: validate):** el Backend no inicializa si las tablas y columnas no existen previamente en la base de datos.

1.4 Justificación de la Migración Arquitectónica: De Servidores Gratuitos a VPS Hostinger

Durante las etapas iniciales de prototipado se emplearon plataformas en la nube bajo esquemas gratuitos (Render para la API, Netlify para los frontends y Supabase para la base de datos). No obstante, para garantizar la calidad, disponibilidad comercial y los estándares de sustentación técnica, se tomó la decisión estratégica de migrar la totalidad de la plataforma a una infraestructura dedicada propia en Hostinger Cloud VPS (Ubuntu 24.04 LTS, IP 2.25.75.54) y dominio institucional drivique.tech, sustentada en las siguientes razones técnicas:

- **Eliminación de la Suspensión por Inactividad (Cold Starts):**El plan gratuito de Hostinger VPS colocaba el contenedor en hibernación tras 15 minutos sin peticiones, generando demoras críticas de arranque de entre 50 y 90 segundos para los usuarios. En Hostinger VPS, el microservicio permanece activo 24/7 con respuesta inmediata en milisegundos.
- **Latencia Cero entre API y Base de Datos:**La separación geográfica de PostgreSQL introducía sobrecarga de red en cada transacción. Al alojar PostgreSQL 17 Alpine en la misma máquina bajo una red interna de Docker, la latencia de consulta es de 0 ms y se eliminan las limitaciones de desconexión o pausa del proyecto por inactividad.
- **Unificación de Marca y Eliminación de Problemas CORS:**Se sustituyeron las URLs de prueba dispersas (.netlify.app y .onrender.com) por el dominio propio drivique.tech y su subdominio app.drivique.tech, eliminando incidentes de bloqueo de orígenes cruzados y proyectando credibilidad comercial.
- **Seguridad y Control Absoluto de Respaldos:**El puerto de base de datos no se expone a internet. Asimismo, el equipo gestiona copias de seguridad automatizadas mediante scripts shell (backup.sh) con retención de 7 días y compresión nativa.

2. Alcance, Topología y Arquitectura

2.1 Componentes y Artefactos en Alcance

2.1.1 Panel de administración (Caddy 2 CE)

Plataforma web para gestión y monitoreo del ciclo de vida de los contenedores Docker en el servidor VPS, administración de stacks docker-compose, inspección de métricas de CPU/RAM y lectura centralizada de logs.

2.1.2 Capa Frontend Web (React SPA / Hostinger VPS Edge)

Aplicación Single Page construida con React 19, Vite, Tailwind CSS 4, React Router 7 y Zustand. Desplegada en Hostinger VPS (drivique-web.netlify.app), cuenta con redirecciones SPA (/* -> /index.html 200) e integración con OAuth Google/Facebook y checkout Wompi Sandbox.

2.1.3 Capa Frontend Móvil (React Native / Expo en Hostinger VPS App)

Aplicación móvil desarrollada con Expo SDK 52 y React Native. Para visualización web y pruebas se distribuye en Hostinger VPS (app.drivique.tech), consumiendo el mismo backend mediante variables con prefijo EXPO_PUBLIC_.

2.1.4 Capa Backend API (Java Spring Boot 4.0.8)

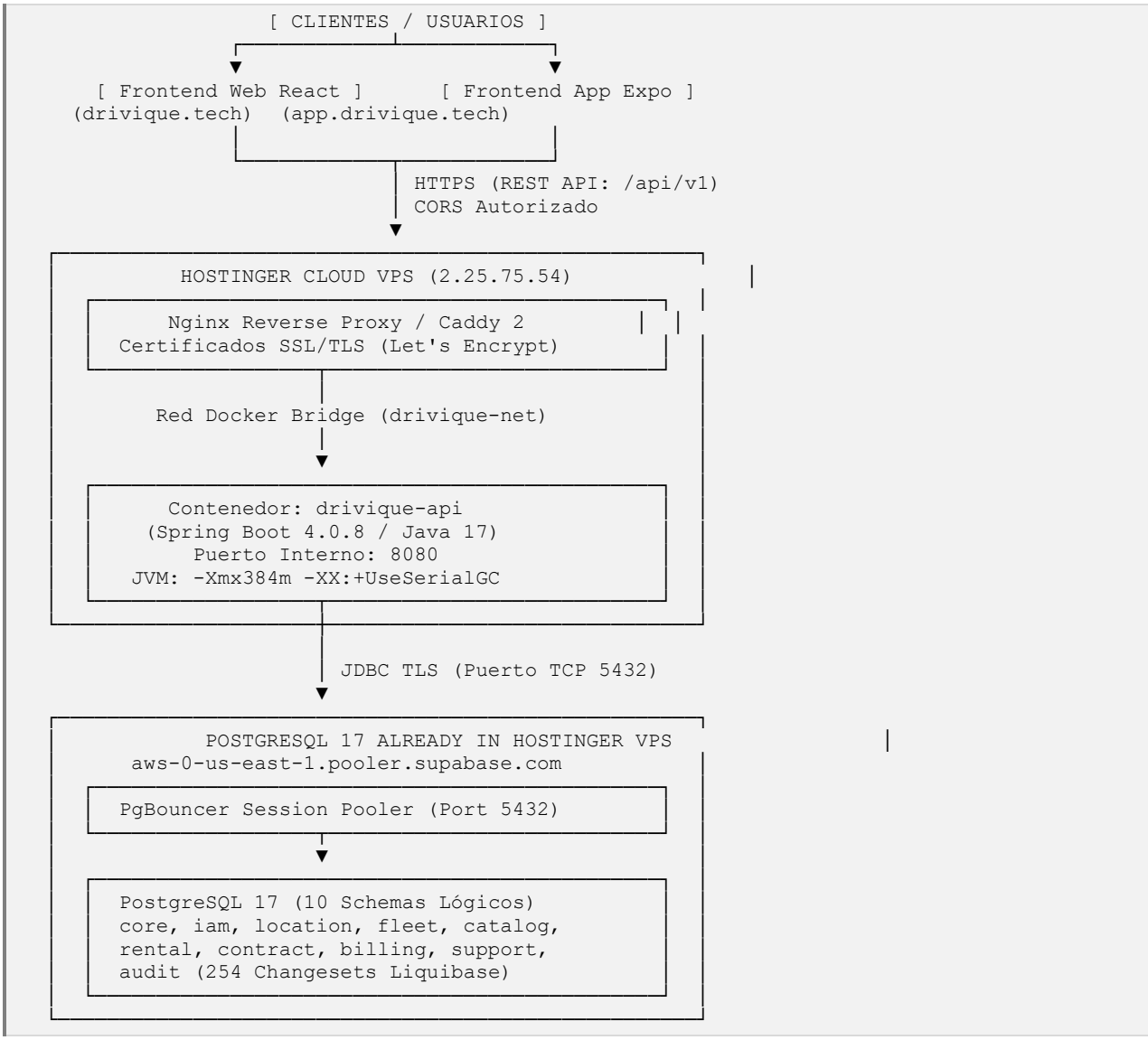
Microservicio REST desarrollado en Java 17 con Spring Boot 4.0.8, Spring Security con tokens JWT, Spring Data JPA para persistencia y Spring Boot Actuator para health checks. Desplegado como contenedor Docker en Hostinger VPS y VPS Staging.

2.1.5 Capa de Persistencia (PostgreSQL en PostgreSQL)

Instancia de base de datos relacional PostgreSQL 17 gestionada en PostgreSQL (AWS us-east-1). Cuenta con PgBouncer Session Pooler en el puerto 5432 para garantizar conectividad IPv4 estable con bajo consumo de conexiones TCP.

2.2 Diagrama de Arquitectura de Red e Infraestructura

El siguiente diagrama esquematiza la topología de red y la arquitectura contenerizada desplegada en producción sobre el servidor Hostinger Cloud VPS, detallando el flujo de tráfico HTTPS/HTTP3, la terminación segura de certificados SSL mediante Caddy 2 y el aislamiento de microservicios y persistencia en la red privada Docker Bridge:



2.3 Matriz de Ambientes, Direccionamiento IP y Puertos

Servicio	Ambiente	Host / Dominio Público	Red Interna	Puerto Interno	Puerto Expuesto
Caddy 2 CE	VPS Staging	https://2.25.75.54:443	host	9000 / 9443	9443 (HTTPS)
Frontend Web	Hostinger VPS Prod	https://drivique.tech	CDN Edge	N/A	443 (HTTPS)
Frontend App	Hostinger VPS Prod	https://app.drivique.tech	CDN Edge	N/A	443 (HTTPS)
Backend API	Hostinger VPS / VPS	https://drivique.tech	drivique-net	8080	443 / 8080
PostgreSQL BD	PostgreSQL Cloud	2.25.75.54 (Hostinger VPS)	TCP SSL	5432	5432 (TCP)

3. Requisitos Previos y Preparación (Pre-deployment)

3.1 Checklist de Prerrequisitos del Sistema Operativo y VPS

3.1.1 Recursos de hardware disponibles

- **CPU:** mínimo 2 núcleos vCPU.
- **Memoria RAM:** mínimo 2 GB (recomendado 4 GB para soportar compilación Docker y JVM).
- **Almacenamiento:** mínimo 20 GB SSD NVMe disponibles.

3.1.2 Estado del motor Docker y socket local

- Motor Docker Engine 26.x o superior en ejecución (systemctl status docker o Docker Desktop Engine running).
- Docker Compose V2 disponible (docker compose version).
- Socket Docker activo en /var/run/docker.sock.

3.2 Estrategia y Ejecución de Respaldos (Backups)

3.2.1 Respaldo lógico de base de datos (Dump / Snapshot)

Extracción de respaldo lógico en PostgreSQL antes de aplicar cambios:

```
pg_dump -h 2.25.75.54 (Hostinger VPS) -p 5432 \
        -U drivique_migrator / drivique_app -d postgres \
        -F c -b -v -f backup_drivique_pre_deploy.dump
```

3.2.2 Respaldo de volúmenes persistentes de Docker

Backup del volumen de uploads y documentos KYC:

```
docker run --rm -v drivique_uploads:/data -v $(pwd):/backup alpine \
       tar czf /backup/uploads_backup.tar.gz /data
```

3.2.3 Respaldo de configuraciones de Stacks y variables

Copia de seguridad de los archivos .env de todos los componentes en bóveda segura.

3.3 Gestión de Variables de Entorno y Secretos

3.3.1 Matriz de Variables del Backend API (Hostinger VPS / VPS)

Variable	Valor Configurado	Propósito
PORT	8080	Puerto HTTP interno del microservicio.

Variable	Valor Configurado	Propósito
SPRING_PROFILES_ACTIVE	dev	Perfil de configuración activo de Spring Boot.
DB_URL	jdbc:postgresql://2.25.75.54 (Hostinger VPS):5432/postgres?sslmode=require	Cadena JDBC apuntando a PostgreSQL PgBouncer con SSL.
DB_USERNAME	drivique_migrator / drivique_app	Usuario del pooler de PostgreSQL.
DB_PASSWORD	Protegida en deploy/.env (600)	Contraseña del pooler de PostgreSQL.
CORS_ALLOWED_ORIGINS	https://drivique.tech,https://app.drivique.tech,https://6ad0ca4d94879b3379a63ea9--drivique.tech	Lista blanca de orígenes permitidos (Web, App y Previews).
SPRING_MAIL_HOST	smtp.gmail.com	Servidor SMTP para envío de correos transaccionales.
SPRING_MAIL_PORT	587	Puerto STARTTLS de Gmail.
SPRING_MAIL_USERNAME	odrivique@gmail.com	Cuenta de correo transaccional.
SPRING_MAIL_PASSWORD	pgvfqqoivraeppdd	Contraseña de aplicación de 16 caracteres de Google.
APP_AUTH_MAIL_FROM	odrivique@gmail.com	Remitente de correos de activación y reset de clave.
JAVA_TOOL_OPTIONS	-Xmx384m -XX:+UseSerialGC	Optimización de memoria RAM para contenedores.

3.3.2 Matriz de Variables del Frontend Web (Hostinger VPS - drivique-web)

Clave (Key)	Valor	Propósito

Clave (Key)	Valor	Propósito
VITE_API_URL	https://drivique.tech/api/v1	URL base de la API REST del backend en la nube.
VITE_GOOGLE_CLIENT_ID	18960724578-h53pr526uva5mtb9doup86f5hjei231c.apps.googleusercontent.com	ID de cliente de Google Identity Services.
VITE_FACEBOOK_APP_ID	1072551762252390	Identificador de aplicación de Facebook Login SDK.

3.3.3 Matriz de Variables del Frontend Móvil (Hostinger VPS - drivique-app)

Clave (Key)	Valor	Propósito
EXPO_PUBLIC_API_URL	https://drivique.tech/api/v1	URL de conexión al backend para la aplicación Expo.
EXPO_PUBLIC_GOOGLE_CLIENT_ID	18960724578-h53pr526uva5mtb9doup86f5hjei231c.apps.googleusercontent.com	ID de cliente OAuth de Google para entorno Expo.
EXPO_PUBLIC_FACEBOOK_APP_ID	1072551762252390	ID de aplicación de Facebook para entorno Expo.

3.3.4 Configuración de Pasarela de Pago Wompi (Sandbox)

En ambas interfaces frontend (Web y App), las credenciales de prueba se encuentran configuradas de forma aislada en el código (config.json y wompiService.ts), por lo cual no requieren inyección de secretos adicionales:

Parámetro	Valor
Ambiente	sandbox
Public Key	pub_test_Xq9JLHZllsreCdWcFQoubBvUZUQ61sFC
Integrity Key	test_integrity_atd8mLC87VadjQvpBEZASMdr9HfUTyP7

Parámetro	Valor
Moneda Base	COP

3.4 Protocolo de Comunicación y Ventana de Mantenimiento

- Notificación previa al equipo con 24 horas de antelación sobre la ventana operativa.
- Despliegue de página temporal de mantenimiento en el proxy si aplica.
- Monitoreo constante del canal de despliegue en Discord/Slack durante la ejecución.

4. Procedimiento de Despliegue Paso a Paso (Runbook)

4.1 Verificación y Acceso a la Plataforma Base (Caddy 2 CE)

1. Acceder al portal web de Caddy 2 en https://<IP_VPS>:9443.
2. Iniciar sesión con credenciales administrativas (admin).
3. Seleccionar el ambiente primario Local / Primary Environment.
4. Verificar que el estado del clúster muestre el Engine activo y suficiente memoria disponible.

4.2 Despliegue y Migración de Base de Datos

4.2.1 Ejecución de scripts DDL/DML

Opción A (Liquibase CLI): ejecutar el contenedor de Liquibase para aplicar los 254 changesets:

```
docker run --rm -v "${PWD}:/workspace" -w /workspace liquibase/liquibase:4.29 `
  --url="jdbc:postgresql://2.25.75.54 (Hostinger VPS):5432/postgres?sslmode=require" `
  --username="drivique_migrator / drivique_app" `
  --password="Protegida en deploy/.env (600)" `
  --changelog-file="changelog/db.changelog-master.yaml" update
```

Opción B (Script Maestro SQL): cargar y ejecutar el script database_drivique.sql directamente desde el SQL Editor del panel de PostgreSQL.

4.2.2 Verificación de integridad

1. Acceder a PostgreSQL Dashboard > Table Editor.
2. Confirmar que existan los 10 esquemas relacionales (core, iam, location, fleet, catalog, rental, contract, billing, support, audit).
3. Verificar la tabla databasechangelog con 254 registros en estado EXECUTED y las 632 filas de datos semilla.

4.3 Despliegue y Orquestación del Stack de Aplicación

4.3.1 Corrección y Estandarización del Dockerfile (Backend)

Para solventar el error de compilación ERROR: lstat /target: no such file or directory (ocasionado porque target/ no existe en Git), se implementa la compilación multietapa:

```
# Etapa 1: Compilar la aplicación con Maven y JDK 17
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Etapa 2: Imagen ligera de ejecución JRE Alpine
FROM eclipse-temurin:17-jre-alpine
```

```
WORKDIR /app
COPY --from=build /app/target/drivique-api-*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

4.3.2 Carga del archivo docker-compose.yml en Caddy 2

En Caddy 2, ir a Stacks > Add stack con el nombre drivique-stack. Contenido del Compose:

```
version: '3.8'
services:
  drivique-api:
    build:
      context: ./Back-end
      dockerfile: Dockerfile
    container_name: drivique-api
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      PORT: 8080
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE}
      DB_URL: ${DB_URL}
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}
      CORS_ALLOWED_ORIGINS: ${CORS_ALLOWED_ORIGINS}
      SPRING_MAIL_HOST: ${SPRING_MAIL_HOST}
      SPRING_MAIL_PORT: ${SPRING_MAIL_PORT}
      SPRING_MAIL_USERNAME: ${SPRING_MAIL_USERNAME}
      SPRING_MAIL_PASSWORD: ${SPRING_MAIL_PASSWORD}
      JAVA_TOOL_OPTIONS: ${JAVA_TOOL_OPTIONS}
    networks:
      - drivique-net
networks:
  drivique-net:
    driver: bridge
```

4.3.3 Despliegue en Hostinger VPS Cloud

1. En Hostinger VPS Dashboard, crear New Web Service conectado al repositorio project-drivique/Back-end.
2. Configurar los parámetros: Name: drivique-api; Language: Docker; Branch: main (o HU-INT-21-main); Root Directory: (en blanco).
3. En Environment Variables, pegar el bloque de variables de la sección 3.3.1.
4. Hacer clic en Deploy web service.

4.4 Configuración de Enrutamiento, Dominio y Certificados SSL/TLS

4.4.1 Despliegue del Frontend Web en Hostinger VPS

1. Conectar el repositorio project-drivique/front-end_web.
2. Parámetros de compilación: Base directory: web-drivique; Build command: npm run build; Publish directory: dist.

3. Cargar las variables de entorno de la sección 3.3.2 (VITE_API_URL, VITE_GOOGLE_CLIENT_ID, VITE_FACEBOOK_APP_ID).
4. Ir a Deploys > Trigger deploy > Clear cache and deploy site.
5. El certificado SSL/TLS es aprovisionado automáticamente por Hostinger VPS Edge.

4.4.2 Despliegue del Frontend App en Hostinger VPS

1. Conectar el repositorio project-drivique/front-end_movil.
2. Parámetros de compilación: Base directory: app-drivique; Build command: npx expo export -p web (o npm run build); Publish directory: dist.
3. Inyectar las variables de la sección 3.3.3 (EXPO_PUBLIC_API_URL, EXPO_PUBLIC_GOOGLE_CLIENT_ID, EXPO_PUBLIC_FACEBOOK_APP_ID).
4. Ejecutar Clear cache and deploy site.

5. Plan de Verificación, Pruebas y Evidencias (Smoke Tests)

5.1 Matriz de Enlaces y Endpoints de Verificación de Servicios

Componente	Endpoint / URL de Prueba	Código Esperado	Criterio de Aceptación
Caddy 2 CE	https://bos2.hostingervps.com (Consola Web)	200 OK	Consola de administración accesible con login.
Frontend Web	https://drivique.tech	200 OK	Hostinger VPSizado completo sin pantalla blanca. Catálogo interactivo.
Frontend App	https://app.drivique.tech	200 OK	Carga de interfaz móvil en navegador con conexión a la API.
Backend Health	https://drivique.tech/api/actuator/health	200 OK	JSON con estado {"status":"UP","components":{"db":{"status":"UP"}}}
Swagger UI	https://drivique.tech/api/swagger-ui.html	200 OK	Documentación interactiva OpenAPI accesible y funcional.
Conexión BD	TCP Port 5432 PostgreSQL	Conectado	Conexión JDBC exitosa con pool Hikari activo.

5.2 Pruebas de Humo Funcionales (Smoke Tests)

5.2.1 Flujo de autenticación y emisión de tokens

- Petición POST a /api/v1/auth/login con credenciales de prueba (cliente@drivique.com / Cliente123*).
- Validación de respuesta HTTP 200 y retorno de Access Token JWT válido.
- Verificación de botón de Google Identity Services y Facebook Login activos sin errores de client_id.

5.2.2 Pruebas de lectura y consulta de datos

- Petición GET a /api/v1/vehicles y /api/v1/branches.
- Verificación de renderizado de vehículos en la Web (drivique.tech) y en la App (app.drivique.tech).

5.2.3 Pruebas de escritura y persistencia transaccional

- Creación de solicitud de reserva desde el formulario web.
- Verificación de persistencia inmediata en la tabla rental.reservations.
- Apertura exitosa del widget de Wompi Sandbox con referencia RES-XXXX y monto en centavos COP.

5.3 Inspección de Logs y Monitoreo de Recursos

5.3.1 Revisión de trazas de inicio sin excepciones críticas

Inspeccionar los logs del backend en Hostinger VPS / Caddy 2 y confirmar:

- Tomcat started on port 8080 (http) with context path '/api'
- HikariPool-1 - Start completed.
- Ausencia total de CannotCreateTransactionException, Table not found o fallos de validación Hibernate.
- Mensaje final: Started DriviqueApiApplication in X.XXX seconds.

5.3.2 Monitoreo de consumo de CPU y memoria RAM

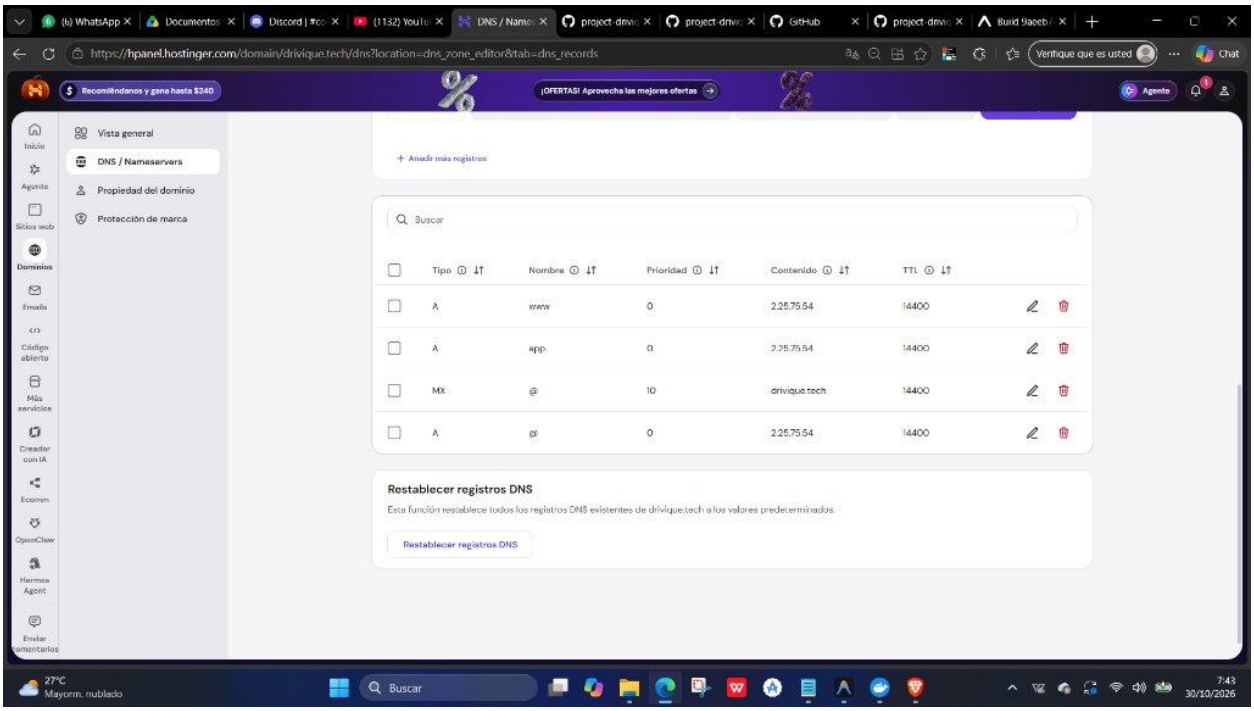
En métricas de Hostinger VPS / Caddy 2, confirmar estabilización de RAM entre 250 MB y 380 MB, respetando la restricción -Xmx384m.

5.4 Registro de Evidencias Visuales de Producción (Screenshots)

A continuación se consolidan las capturas de pantalla que certifican la puesta en producción integral de la plataforma Drivique en el servidor VPS de Hostinger, el enrutamiento de red y DNS, la orquestación de contenedores Docker, la disponibilidad de las interfaces Web y Móvil bajo HTTPS, y la operatividad de los microservicios backend junto a la base de datos PostgreSQL.

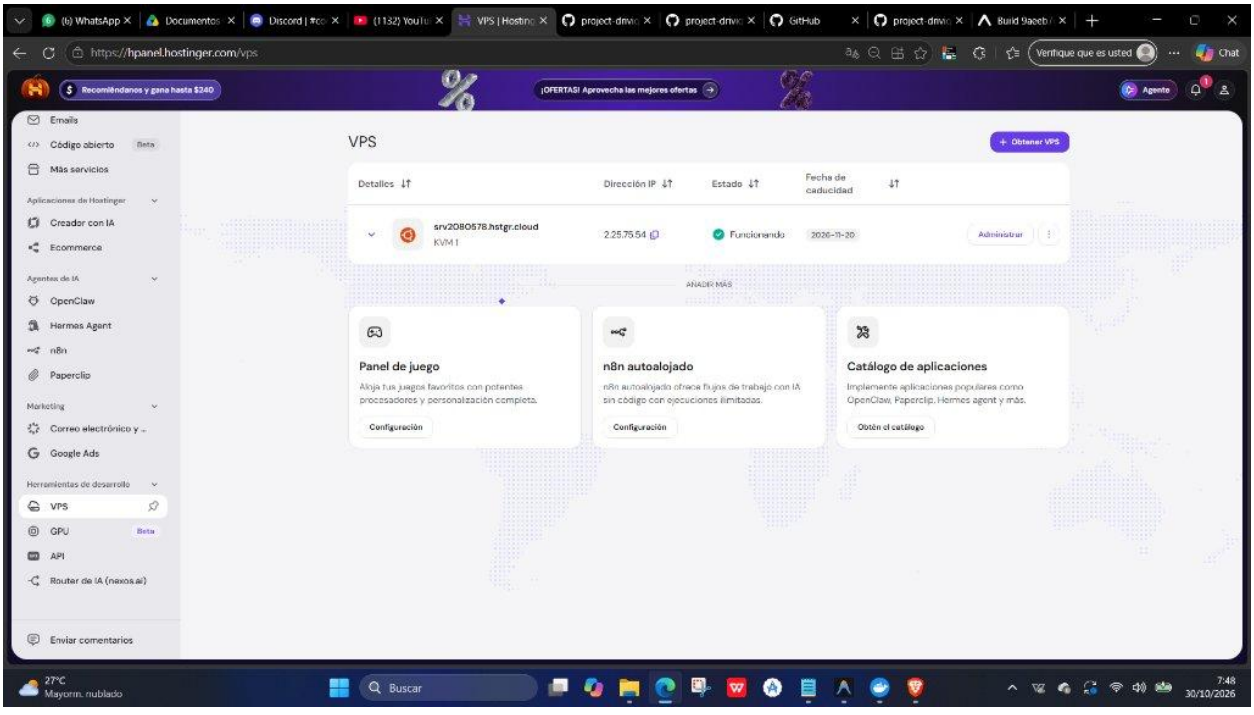
Evidencia 1: Enrutamiento y Administración de Registros DNS en Hostinger hPanel

Configuración oficial de los registros DNS de tipo A para el dominio raíz (drivique.tech), el subdominio móvil (app.drivique.tech) y el alias canónico (www.drivique.tech), todos vinculados a la dirección IPv4 pública 2.25.75.54 del servidor VPS.



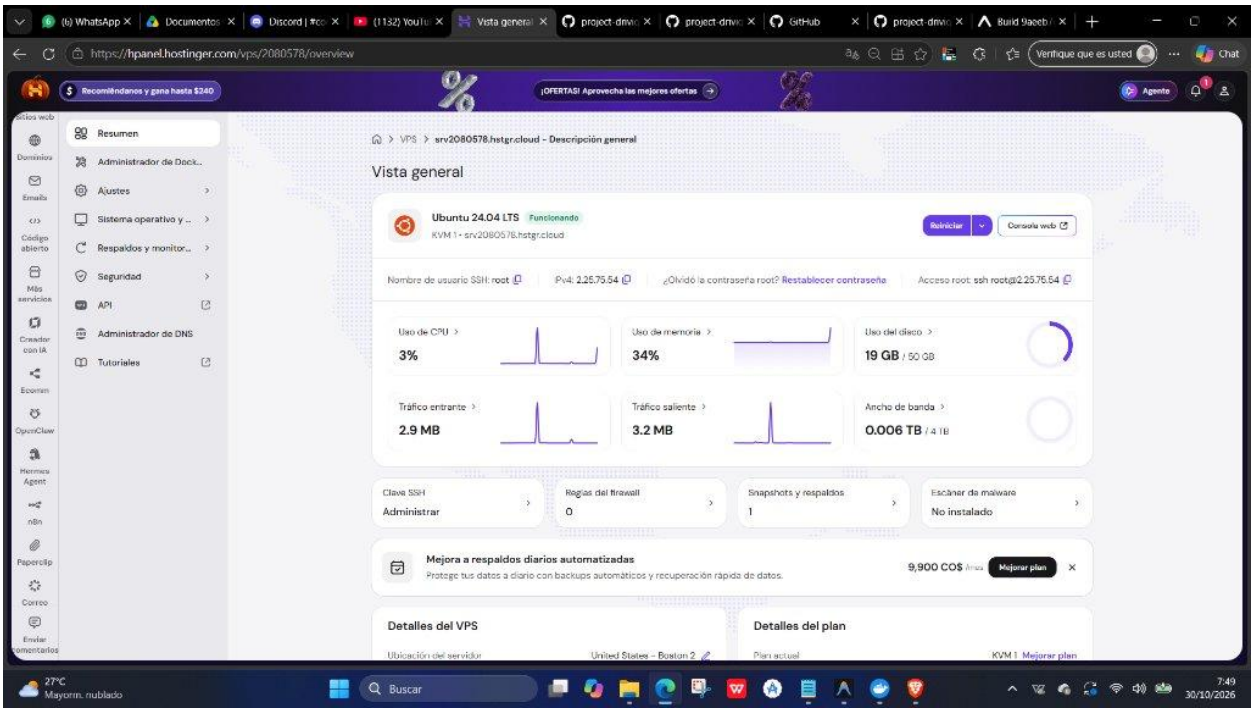
Evidencia 2: Aprovisionamiento y Estado Operativo del Servidor VPS en Hostinger

Vista del panel de servidores de Hostinger certificando el estado operativo "Funcionando" del servidor VPS KVM 1 (srv2080578.hstgr.cloud) con sistema operativo Ubuntu 24.04 LTS y dirección IP pública 2.25.75.54.



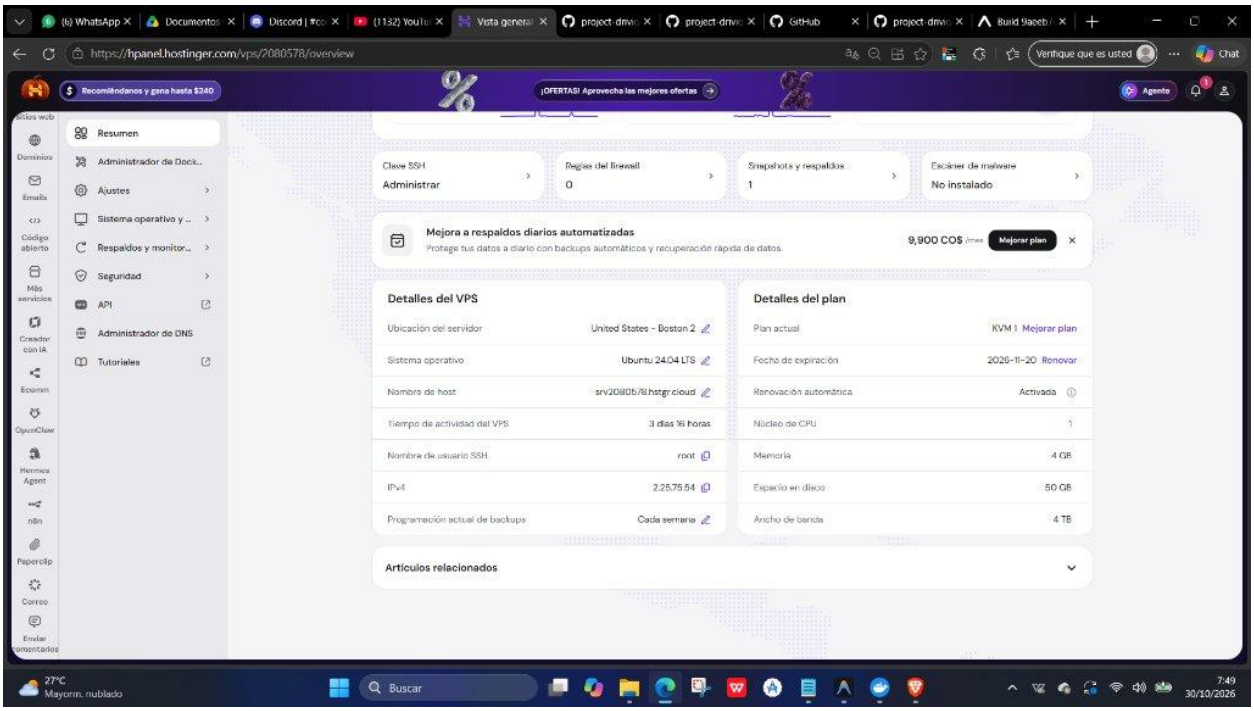
Evidencia 3: Monitoreo de Recursos y Rendimiento en Tiempo Real del VPS

Panel de métricas en producción evidenciando un desempeño altamente eficiente: Uso de CPU al 3%, uso de memoria RAM al 34% y almacenamiento en disco en 19 GB / 50 GB con la pila completa de microservicios contenerizados activa.



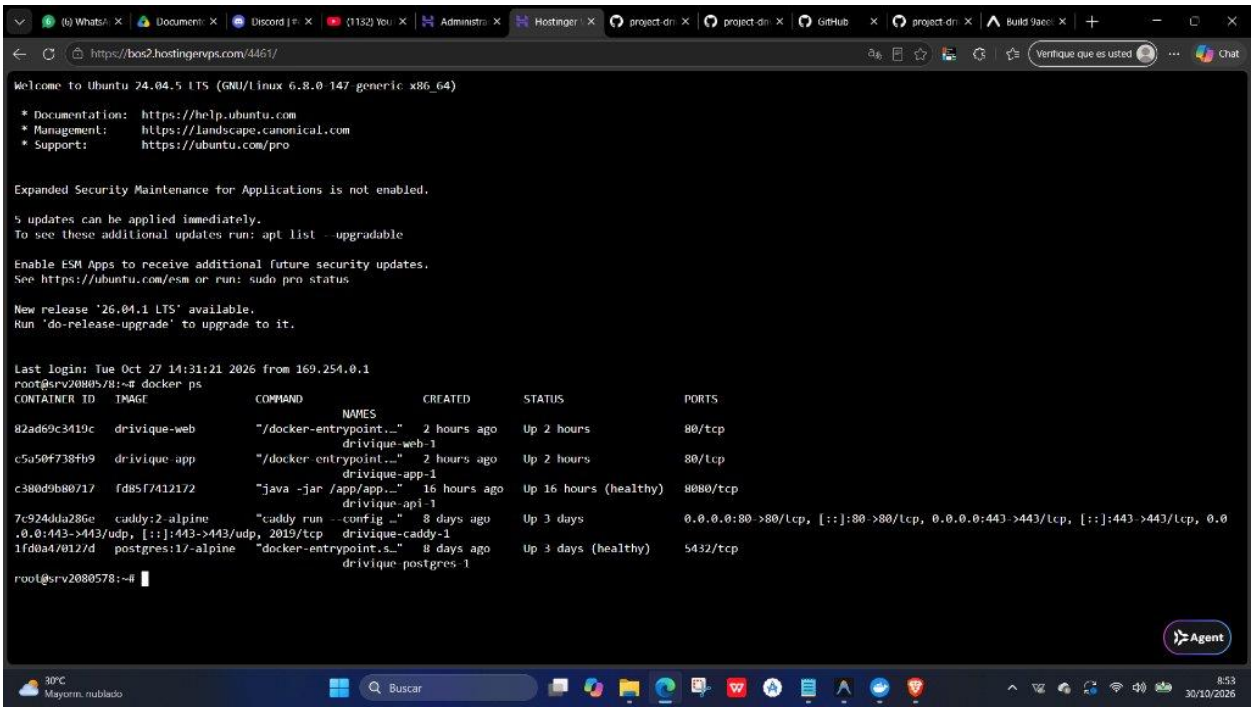
Evidencia 4: Especificaciones Técnicas de Hardware y Conectividad del Host

Ficha técnica del VPS en el centro de datos Boston 2 (EE. UU.): 1 núcleo vCPU, 4 GB de memoria RAM, 50 GB de almacenamiento SSD NVMe y acceso de administración remota segura vía SSH (ssh root@2.25.75.54).



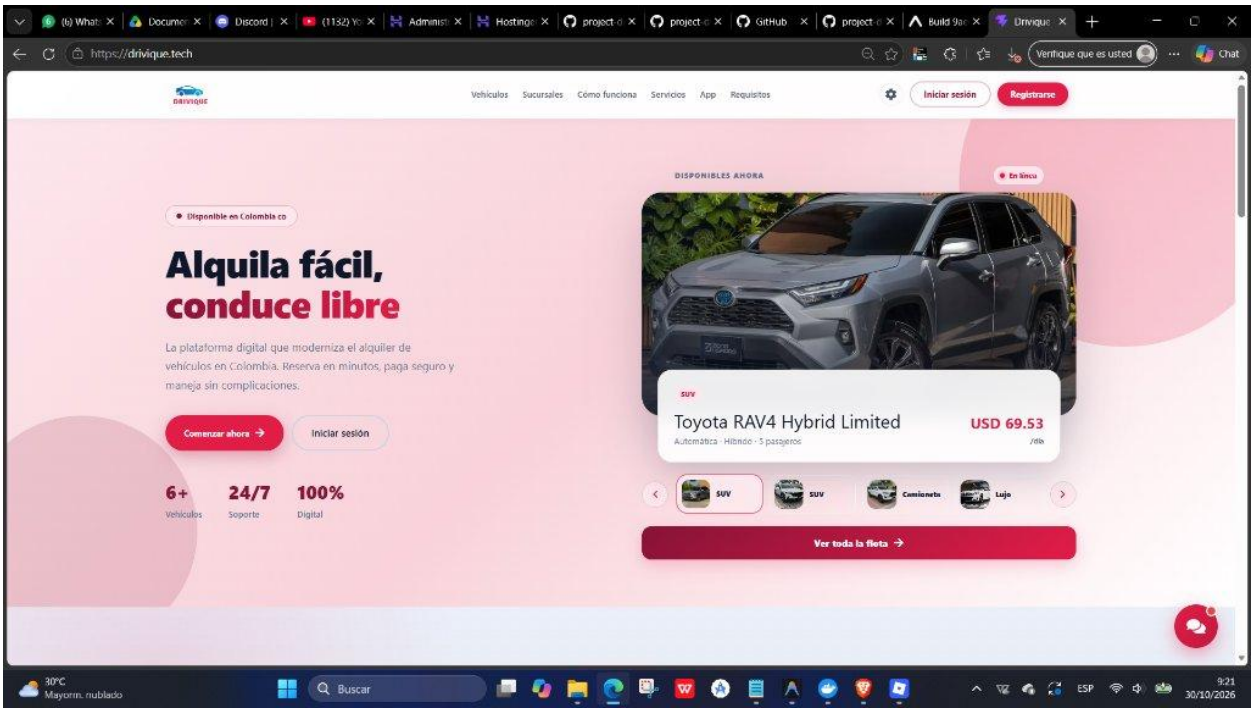
Evidencia 5: Orquestación de Contenedores Docker en Consola Web (docker ps)

Terminal del servidor Ubuntu ejecutando el comando docker ps, demostrando los 5 contenedores activos y saludables: drivique-caddy-1 (puertos 80/443 con TLS), drivique-api-1 (Spring Boot healthy en puerto 8080), drivique-web-1 (puerto 80), drivique-app-1 (puerto 80) y drivique-postgres-1 (PostgreSQL 17 healthy en 5432).



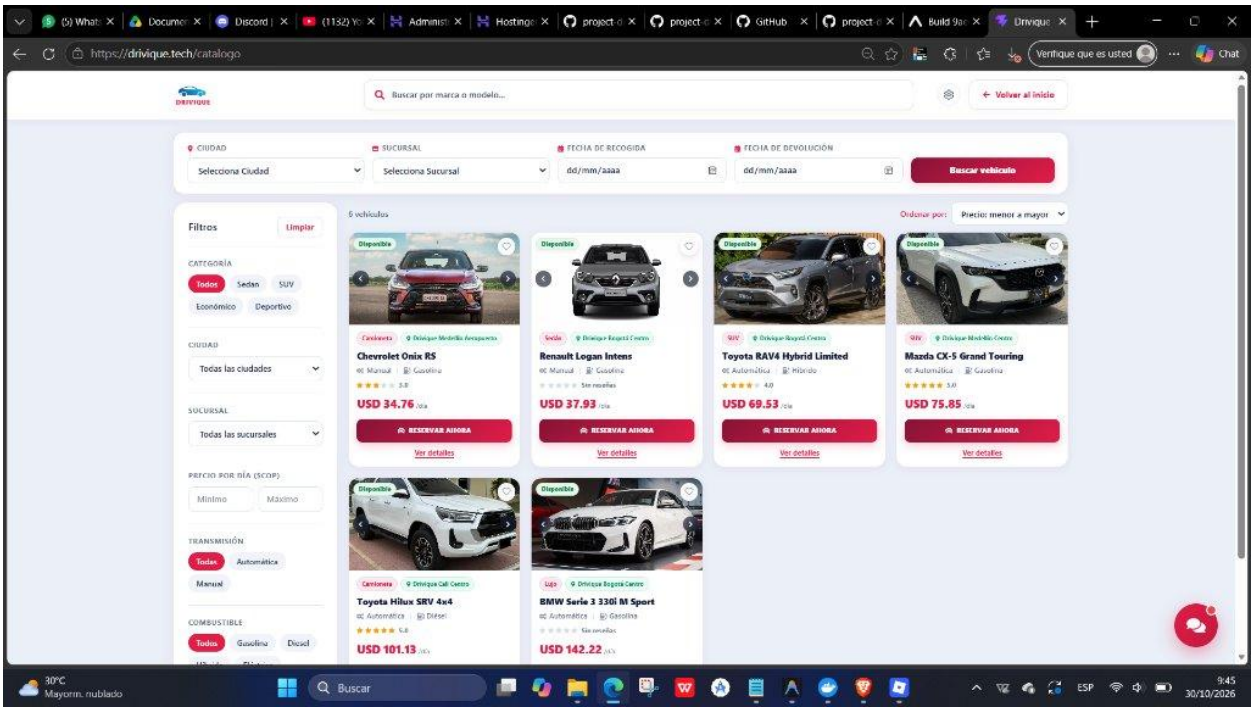
Evidencia 6: Puesta en Producción del Frontend Web (https://drivique.tech)

Navegación en vivo sobre la plataforma web de Drivique, validando el cifrado HTTPS provisto automáticamente por Caddy y la correcta carga del portal principal con navegación y autenticación en drivique.tech.



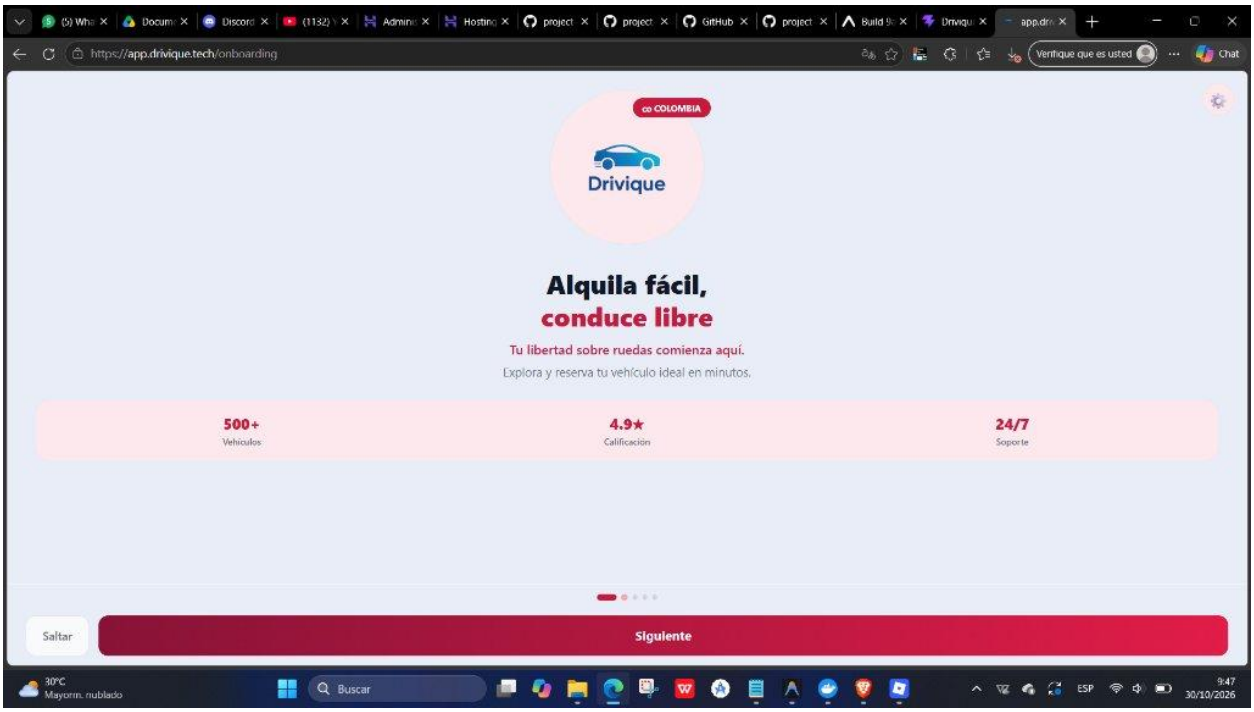
Evidencia 7: Catálogo Interactivo de Vehículos y Persistencia Transaccional

Módulo de catálogo en producción (https://drivique.tech/catalogo) consumiendo datos en tiempo real desde PostgreSQL mediante la API REST de Spring Boot, con filtrado dinámico por categoría, ciudad y tarifas.



Evidencia 8: Frontend Móvil Multiplataforma en Producción (https://app.drivique.tech)

Despliegue de la aplicación desarrollada en React Native con Expo SDK 52 exportada para entorno web, operando de forma autónoma en el subdominio seguro https://app.drivique.tech.



Evidencia 9: Validación de Certificado de Seguridad SSL/TLS en Subdominio Móvil

Detalle de la barra de direcciones del navegador comprobando la validez del candado de seguridad HTTPS (TLS 1.3) emitido de forma automatizada por Let's Encrypt para el subdominio https://app.drivique.tech.



Evidencia 10: Verificación de Endpoints de la API REST y Conexión a Base de Datos

Respuesta HTTP 200 OK en formato JSON emitida por el microservicio Spring Boot al consultar https://drivique.tech/api/v1/mileage-plans, certificando la conexión exitosa del datasource JDBC hacia PostgreSQL.

